

School of Computer Science and Artificial Intelligence

Lab Assignment # 13.2

Program	: B. Tech (CSE)
Specialization	: -
Course Title	: AI Assisted Coding
Course Code	: 23CS002PC304
Semester	: II
Academic Session	: 2025-2026
Name of Student	: G.Sampath Kumar
Enrollment No.	: 2403A51L36
Batch No.	: 52
Date	: 10/03/26

Submission Starts here**Screenshots:****Task Description -1 (Refactoring – Eliminating Repetitive Logic)****Prompt:**

Refactor the following Python program to eliminate repetitive logic.

Requirements:

- 1. Identify repeated calculations in the code.**
- 2. Encapsulate the repeated logic into a reusable function.**
- 3. The function should compute both area and perimeter of a rectangle.**
- 4. Ensure the output remains exactly the same as the original program.**
- 5. Add proper Python docstrings to the function explaining parameters and return values.**
- 6. Keep the code clean, readable, and maintainable.**

Here is the original code:

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

```
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Output:

```
1 def rectangle_properties(length, width):
2     """Calculate the area and perimeter of a rectangle.
3
4     Parameters
5     -----
6     length : float or int
7         The length of the rectangle.
8     width : float or int
9         The width of the rectangle.
10
11     Returns
12     -----
13     tuple
14         A tuple of two values: (area, perimeter).
15     """
16     area = length * width
17     perimeter = 2 * (length + width)
18     return area, perimeter
19
20
21 length = 8
22 width = 4
23 area, perimeter = rectangle_properties(length, width)
24 print("Area:", area)
25 print("Perimeter:", perimeter)
26
27 length = 12
28 width = 6
29 area, perimeter = rectangle_properties(length, width)
30 print("Area:", area)
31 print("Perimeter:", perimeter)
32
```

Task Description -2 (Refactoring – Modularizing Inline Calculations)**Prompt:**

Refactor the following Python code by extracting repeated inline calculations into a reusable function.

Requirements:

1. Detect repeated calculation patterns.
2. Create a reusable function to compute the final amount after applying discount.
3. Add a clear docstring explaining the function purpose, parameters, and return value.
4. Ensure the final output remains the same as the original program.
5. Improve readability and modularity.

Original Code:

```
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
```

```
amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
```

Output:

```
1 def calculate_final_amount(amount, discount_rate=0.10):
2     """
3     Calculate the final amount after applying a discount.
4
5     Parameters:
6     amount (float or int): The original amount before discount.
7     discount_rate (float): The discount rate to apply (e.g., 0.10 for 10%).
8
9     Returns:
10    float: The final amount after subtracting the discount.
11    """
12    discount = amount * discount_rate
13    return amount - discount
14
15
16 amount = 1000
17 final_amount = calculate_final_amount(amount)
18 print("Final Amount:", final_amount)
19
20 amount = 2000
21 final_amount = calculate_final_amount(amount)
22 print("Final Amount:", final_amount)
23
```

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

Prompt:

Optimize the following Python program to improve performance.

Requirements:

1. Identify inefficient nested loops and repeated conditional checks.
2. Replace the inefficient lookup logic with a more efficient data structure such as a set.
3. Maintain the same program output.
4. Provide both the original and optimized versions.
5. Measure and compare execution time of both versions using the time module.
6. Add comments explaining the optimization.

Original Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
```

```
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True

    if exists:
        print(name, "found")
    else:
        print(name, "not found")
```

Output:

```

1 import time
2 # Original code (baseline version)
3 students = ["Anil", "Bhavya", "Charan", "Divya"]
4 check_list = ["Charan", "Esha", "Bhavya"]
5
6 def original_check(students_list, check_list_local):
7     """
8     Original implementation using nested loops.
9     Time complexity: O(len(students_list) * len(check_list_local))
10    """
11    for name in check_list_local:
12        exists = False
13        for student in students_list:
14            if name == student:
15                exists = True
16        if exists:
17            print(name, "found")
18        else:
19            print(name, "not found")
20
21    # Optimized code using set lookups
22    # Optimization:
23    # - Convert the students list to a set once.
24    # - Membership check 'name in students_set' is O(1) on average.
25    # - This removes the inner loop and repeated conditional checks.
26    students_set = set(str)(students) # Precompute for faster membership tests
27
28    def optimized_check(students_set_local, check_list_local):
29        """
30        Optimized implementation using a set for O(1) lookups.
31        Time complexity: O(len(students_set_local) + len(check_list_local))
32        """
33        for name in check_list_local:
34            if name in students_set_local: # Direct, efficient membership check
35                print(name, "found")
36            else:
37                print(name, "not found")
38
39    if __name__ == "__main__":
40        print("Original Version Output:")
41        original_check(students, check_list)
42
43        print("\nOptimized Version Output:")
44        optimized_check(students_set, check_list)
45
46        # Timing comparison
47        # To see a measurable difference, run each version many times.
48        ITERATIONS = 10000
49
50        start = time.time()
51        for _ in range(ITERATIONS):
52            original_check(students, check_list)
53        original_time = time.time() - start
54
55        start = time.time()
56        for _ in range(ITERATIONS):
57            optimized_check(students_set, check_list)
58        optimized_time = time.time() - start
59
60        print("\n--- Timing Comparison ---")
61        print("Original version time :", original_time, "seconds")
62        print("Optimized version time:", optimized_time, "seconds")

```

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

Prompt:

Refactor the following Python code to replace hardcoded numeric values with named constants.

Requirements:

1. Identify all hardcoded numeric values.
2. Create descriptive constants at the top of the program.
3. Use constant names like RATE and TIME instead of raw values.
4. Ensure the program output remains unchanged.
5. Improve code readability and maintainability.
6. Add comments explaining the constants.

Original Code:

```

print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)

```

Output:

```
1  """
2  Program to calculate simple interest for two different principal amounts
3  using descriptive constants instead of hardcoded numeric values.
4  """
5
6  # Principal amounts (in currency units)
7  PRINCIPAL_1 = 1000
8  PRINCIPAL_2 = 2000
9
10 # Simple interest rate (in percent)
11 RATE = 2
12
13 # Time period (in years)
14 TIME = 5
15
16 # Simple Interest = (Principal * Rate * Time) / 100
17 print("Simple Interest:", (PRINCIPAL_1 * RATE * TIME) / 100)
18 print("Simple Interest:", (PRINCIPAL_2 * RATE * TIME) / 100)
19 | Ctrl+L to chat, Ctrl+K to generate
```

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

Prompt:

Improve the readability of the following Python program.

Requirements:

1. Replace unclear variable names with meaningful descriptive names.
2. Add inline comments explaining the logic.
3. Ensure the program output remains exactly the same.
4. Follow Python naming conventions.
5. Make the code easier to understand for beginners.

Original Code:

```
x = 50
y = 30
z = x + y
print(z)
```

Output:

```
1  first_number = 50
2  second_number = 30
3
4  # Add the two numbers together
5  sum_of_numbers = first_number + second_number
6
7  # Print the result (same output as before)
8  print(sum_of_numbers)
9  | Ctrl+L to chat, Ctrl+K to generate
```